



Politecnico di Bari

Dipartimento di Ingegneria Elettrica e dell'Informazione

Corso
Mobile Robotics

Development in ROS2 and Gazebo of a mobile robotic system for autonomous detection and tracking

Docenti:

Prof. Luca De Cicco
Ph.D. Nunzio Barone

Autori:

Walter Amicarelli (598436)
w.amicarelli@studenti.poliba.it

Pantaleo Grosso (598324)
p.grosso1@studenti.poliba.it

Alessandro Pierro (598102)
a.pierro1@studenti.poliba.it

Anno Accademico 2026/2027

Indice

1	Introduction and Project Goals	1
2	Multi-Robot System Architecture and ROS2 Packages	2
2.1	<code>warehouse_gazebo</code> Package	2
2.1.1	Environment logic and operation	2
2.2	<code>turtlebot4</code> Package	3
2.2.1	Kinematic description and integration	3
2.3	<code>warehouse_robot</code> Package	4
2.3.1	Perceptual <i>pipeline</i> and navigation control	4
2.4	<code>fleet_manager</code> Package	5
2.4.1	Decision-making logic and <i>Task Allocation</i>	5
2.5	<code>warehouse_bringup</code> Package	6
2.5.1	<i>Deployment</i> orchestration	6
2.6	<code>fleet_telemetry</code> Package	6
2.6.1	Diagnostic aggregation and visualization	6
3	KPI, Screen Recording and Analysis	7
3.1	Video Demonstrations	7
3.1.1	Patrolling and Interception	7
3.1.2	Detection and Pursuit	8
3.2	Data Analysis	9
3.2.1	KPI1: Capture Time	9
3.2.2	KPI2: Inter-robot Distance	9
3.2.3	KPI3: YOLO Model Accuracy	10
4	Conclusion	11
4.1	Future Developments	11

1 | Introduction and Project Goals

This report describes the design and development in a simulated environment of a multi-agent software architecture dedicated to the autonomous surveillance of a logistic-industrial facility. The operational scenario is based on the use of a fleet (*swarm*), consisting of three mobile robots, modeled on the standard TurtleBot4 hardware.

The primary goal of the project lies in the creation of a firmware capable of detecting, tracking, and intercepting a dynamic target (intruder). The system aims to demonstrate how tactical coordination among multiple autonomous robots can effectively neutralize a threat within a space, overcoming the physical and sensory limits of a single agent. To achieve this general purpose, the work focused on reaching three specific goals:

- **Distributed surveillance:** implementation of coordinated spatial exploration logics. The system ensures that the robots distribute themselves on the map in a geometrically efficient way, maximizing the coverage of the visual area during the patrol routine and avoiding redundant overlaps.
- **Dynamic task allocation:** development of a decision-making module capable of reorganizing the fleet in real time. The system is able to transform a group of robots in independent exploration into a synchronized tactical team, triggered by the detection of the threat (intruder).
- **Resilience and contingency management:** the system is designed to react robustly to adverse operating conditions, such as the temporary loss of visual contact with the intruder.

The achievement of these goals is realized through a global state machine that governs the entire robotic ecosystem, divided into two main operational phases. During the **Nominal Phase**, the robots operate in autonomous mode exploring the environment. At the moment of visual detection of the intruder by a node in the network, the system generates a global alarm trigger: the agent that made the detection begins an autonomous pursuit.

In parallel, the architecture activates the **Tactical Coordination Phase**. A decision-making unit evaluates the current scenario and orchestrates the encirclement maneuver by sending specific directives (*goals*) to the rest of the swarm. The roles are divided as follows: two units converge on the target to force its capture, while the third unit is strategically deployed to guard the only escape route (the entrance door). The entire tactical execution is supervised by the local navigation modules, ensuring safe trajectories (*collision-free*) both towards static obstacles and among the fleet members themselves.

2 | Multi-Robot System Architecture and ROS2 Packages

2.1 warehouse_gazebo Package

The `warehouse_gazebo` software package serves as the primary simulation ecosystem, based on the ROS2 middleware and the Gazebo simulation environment. The goal of this module is to provide a realistic industrial warehouse within which to validate the autonomous navigation, localization, and *target tracking* algorithms. In the context of the software architecture, it provides the essential spatial and physical infrastructure allowing the control systems to operate in realistic conditions before *deployment* on real hardware.

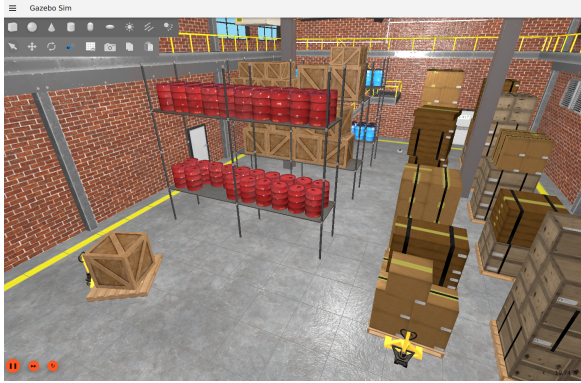


Figure 2.1: Internal view of the warehouse simulation environment.

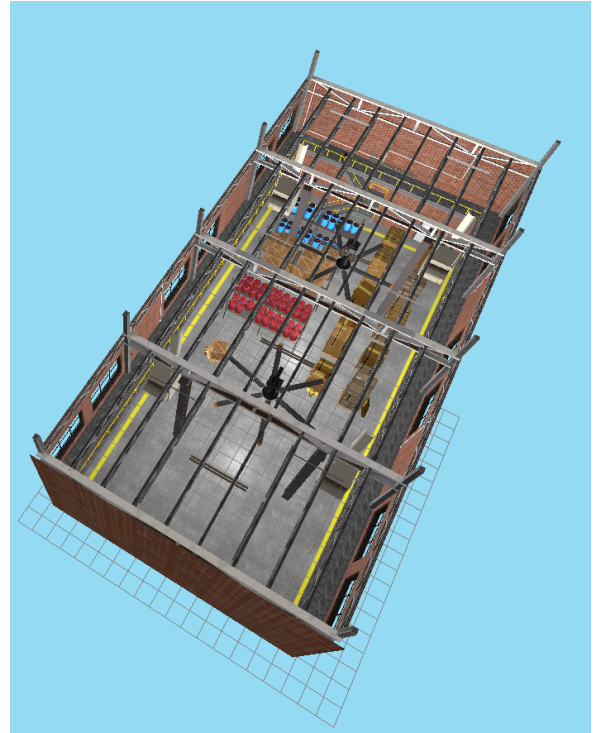


Figure 2.2: Panoramic view of the simulated structure.

2.1.1 Environment logic and operation

From an architectural point of view, the package manages different simulation layers. The lowest level consists of the definition of the physical laws and the 3D topology of the warehouse (models in `SDF` format), which include the management of gravity, friction, and collisions of the actors present in the scenario, including the intruder. Alongside this physical infrastructure is the *deployment* management system, controlled by centralized launch files (such as `launch_warehouse.py`). Instead of starting each single element separately, this software layer automates the configuration and interaction between Gazebo and ROS2, ensuring a controlled and reliable life cycle for the simulated scenario.

Parallel to the 3D physics, the package provides the two-dimensional spatial knowledge base of the system, exposing the *Occupancy Grid* and its related metadata. This information is actively used by the nodes of the navigation stack (Nav2) to perform global localization and *path planning*. Finally, to bridge the gap between physical simulation and algorithmic perception, the package integrates logical modules such as `mock_yolo.py`. The latter implements the *Mock Object* pattern, acting as a fictitious visual perception system capable of detecting intruders in the environment, thus providing the raw data necessary to activate the security system. Ultimately, the `warehouse_gazebo` package faithfully recreates the operation of a robotic system seen as a continuous cycle of interaction with the environment. By integrating the three-dimensional physical world, the static knowledge of the environment (`.pgm` / `.yaml`), and the dynamic extraction of perceptual information, the package forms the foundation on which the robots move and sample sensory data, laying the groundwork for the entire autonomous navigation logic.

2.2 turtlebot4 Package

The architecture provided by the `turtlebot4` package defines the structural, kinematic, and visual representation of the robot. The purpose of this node is to provide high-level algorithms (such as SLAM, navigation, and perception) with the robot's *digital twin*. Without this definition, the system would be unable to calculate spatial transformations (TF *tree*) and manage collisions in the simulation.

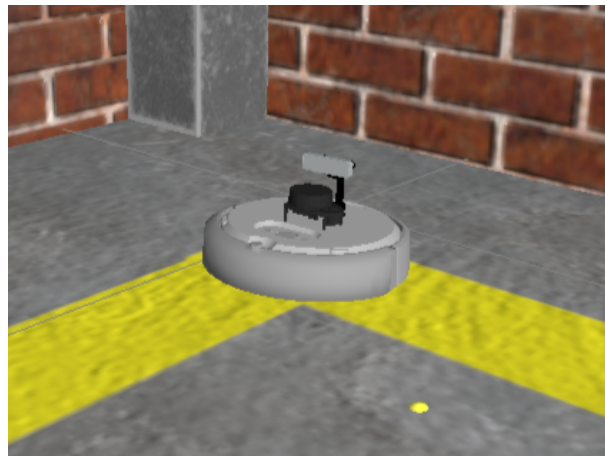


Figura 2.3: Turtlebot4 robot, used as an autonomous mobile agent within the warehouse.

2.2.1 Kinematic description and integration

The functional *core* of the package lies in its ability to model the hardware for the software. This is done by loading the descriptive models of the robot (in URDF or XACRO format) and publishing them as global system parameters through the *Robot State Publisher*. This operation calculates the entire kinematic tree of the vehicle, making the exact position and orientation of the sensors (such as LiDAR and cameras) relative to the base's center of rotation known to all nodes in the network. Furthermore, the package acts as a database for the three-dimensional assets. The structural and sensory *meshes* (such as the OAK-D cameras and the shell) are used on two fronts:

- on the simulation side, they provide Gazebo with the envelopes for collision calculation;
- on the middleware side, they allow tools like RViz to provide high-fidelity visual feedback of the robot's state in real time.

By integrating the 3D models of the chassis and the updating of the kinematic transforms, the `turtlebot4` package masks the low-level hardware details to offer a unified digital representation of the vehicle. This geometric representation is continuously queried by the middleware during operations, which constitutes the spatial prerequisite on which all perception and pursuit maneuvers are based.

2.3 `warehouse_robot` Package

The `warehouse_robot` package constitutes the cognitive and local control level for each agent within the fleet. Operating as a true "onboard brain" (*Edge Computing*), this module manages with total autonomy the navigation stack (Nav2), the computer vision pipeline, and the tactical decision-making logic of the single robot. The design of the package is based on the principle of functional decoupling: entrusting the reactive intelligence to the agent preserves its operational autonomy and frees the central coordinator from demanding calculations, ensuring scalability and robustness for the entire system.

2.3.1 Perceptual *pipeline* and navigation control

The agent's navigation configuration is the result of precise design choices. For localization, the **AMCL** (*Adaptive Monte Carlo Localization*) algorithm based exclusively on LiDAR scanning was adopted. By exploiting the planimetric asymmetries of the warehouse, this choice completely frees up the resources of the optical sensor (OAK-D), dedicating it exclusively to the uninterrupted tracking of the target. At the motion control level, we opted for the **MPPI** (*Model Predictive Path Integral*) predictive controller, which ensures kinematic fluidity by stochastically evaluating thousands of future trajectories, which is essential for crossing narrow passages. The frequencies of the control loop and costmaps were also reduced to prevent CPU saturation during the simulation. In order to ensure high responsiveness, the perception and local control processes operate in parallel. The flow starts from the `yolo_detector` module, which simultaneously processes the RGB and Depth streams to calculate the three-dimensional position of the intruder relative to the camera. The vector obtained in this way is sent in real time to two operational nodes: the `camera_tracker`, which adjusts the mechanical positioning of the sensor, and the `robot_manager`. Built around a Finite State Machine, this latter module manages the onboard tactics (search, pursuit, patrol) and performs the geometric transformation necessary to project the relative position of the intruder within the global map, thus informing the rest of the fleet and the global manager.

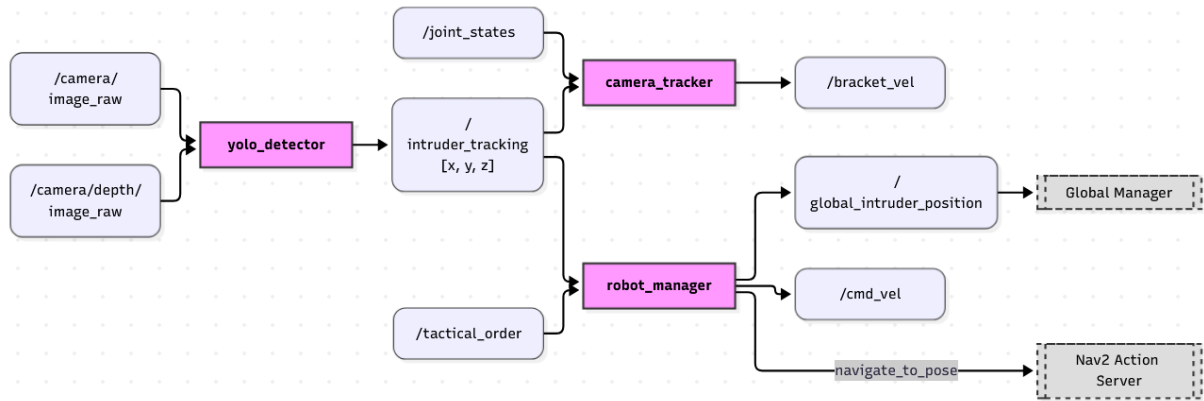


Figura 2.4: Architectural diagram of the perception and local control *pipeline* of the single agent. The data flow illustrates the transformation of raw visual inputs into 3D coordinates.

Fundamental to the multi-agent operation is the implementation of "TF *Sandboxing*". To avoid conflicts in spatial frames when multiple robots share the same topic, the launch files assign a *namespace* to the transform tree of each agent. A dedicated aggregator module (`tf_merged`) then takes care of merging these isolated trees and routing them to the global topics, facilitating the visualization of the simulation on RViz.

2.4 fleet_manager Package

The `fleet_manager` software package constitutes the centralized supervision level of the multi-robot architecture. Its task is to aggregate the distributed state of the fleet, evaluate it, and cyclically translate it into tactical orders to intercept the intruder. This package represents the only node in the system equipped with a global vision and does not deal with kinematics, remaining independent from the hardware platform used.

2.4.1 Decision-making logic and *Task Allocation*

The logical *core* of the supervisor implements a high-level finite state machine that reacts to the events reported by the ROS2 network. When the intruder is detected, the manager transitions into the tactical phase and, to efficiently distribute the interception and encirclement orders, it mathematically solves the optimal task assignment using the **Hungarian algorithm** (*linear sum assignment*), applied to a cost matrix based on the *Manhattan* distance between the current positions of the robots and the dynamic goals (the entrance/exit door and the estimated position of the target). This approach ensures that each agent is directed towards the most advantageous strategic point, minimizing the energy spent and the overall maneuver time of the swarm. Parallel to the task allocation, the manager actively evaluates the mission termination conditions, interrupting the tactical directives as soon as it detects that an agent has reached a critical engagement distance (less than 1.5 m) from the target.

2.5 `warehouse_bringup` Package

In distributed robotics, the orderly startup of components is an architectural problem in itself. The `warehouse_bringup` package fulfills this specific function: defining the coordinated *deployment* of the system. Unlike the other nodes, it is an entirely declarative package, devoid of control logic or Python/C++ executables.

2.5.1 *Deployment* orchestration

The `warehouse_bringup` package takes care of instantiating, in a parameterized and sequenced manner, the entire scenario: it starts the Gazebo *backend*, calls the fleet constructors applying the correct *namespaces* to avoid collisions, initializes the three independent instances of the Nav2 navigation system, and finally boots the global manager. Therefore, the `warehouse_bringup` gathers a set of isolated software packages and disconnected robots into an integrated multi-agent system, that can be started and is ready to use through a single command.

2.6 `fleet_telemetry` Package

Closing the software ecosystem is the `fleet_telemetry` package, designed to cover the role of human-machine interface (HMI). Its purpose is to provide a clear and structured view of the mission's progress, aggregating the telemetry data and the directives of the global supervisor to generate a "*Multi-Agent Tactical Dashboard*".

2.6.1 Diagnostic aggregation and visualization

This node acts as a strictly passive observer. The telemetry module, through subscriptions to the fleet's topics (speeds, distances, status flags), extracts the key variables and formats them in real time into a structured text panel. This process occurs without ever issuing commands, ensuring that the diagnostics do not interfere in any way with the *real-time* integrity of the pursuit control loop.

3 | KPI, Screen Recording and Analysis

3.1 Video Demonstrations

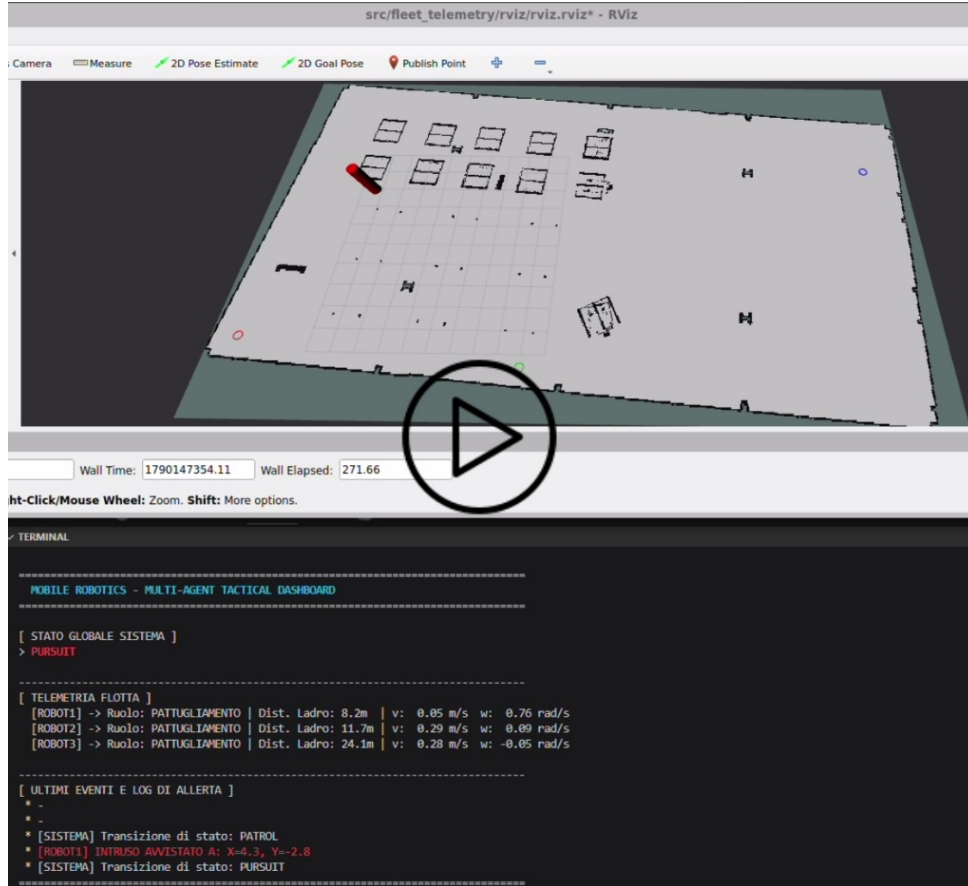


Figura 3.1: Visual demonstration of the patrol and security procedure.

3.1.1 Patrolling and Interception

The video illustrates the operational evolution of the swarm through the RViz interface, starting from the normal surveillance routine. Initially, the *Multi-Agent Tactical Dashboard* at the bottom of the screen confirms the global "PATROL" state (distance from target "N/A"), with the three agents navigating in decentralization to cover the warehouse. The dynamics change upon the intruder's appearance: after an initial phase in which the intruder is not spotted, he enters the visual area of ROBOT1, triggering the security procedure. The global state instantly transitions to "TACTICAL" and the supervisor reorganizes the fleet by assigning specific tasks. ROBOT1 is assigned the strategic guarding of the door to block the escape route, while ROBOT2 and ROBOT3 converge in an active pursuit on the thief's coordinates. The demonstration successfully concludes when ROBOT2 breaks the engagement threshold of 1.5 m from the target, triggering the final "OBJECTIVE REACHED" state and stopping the simulation.

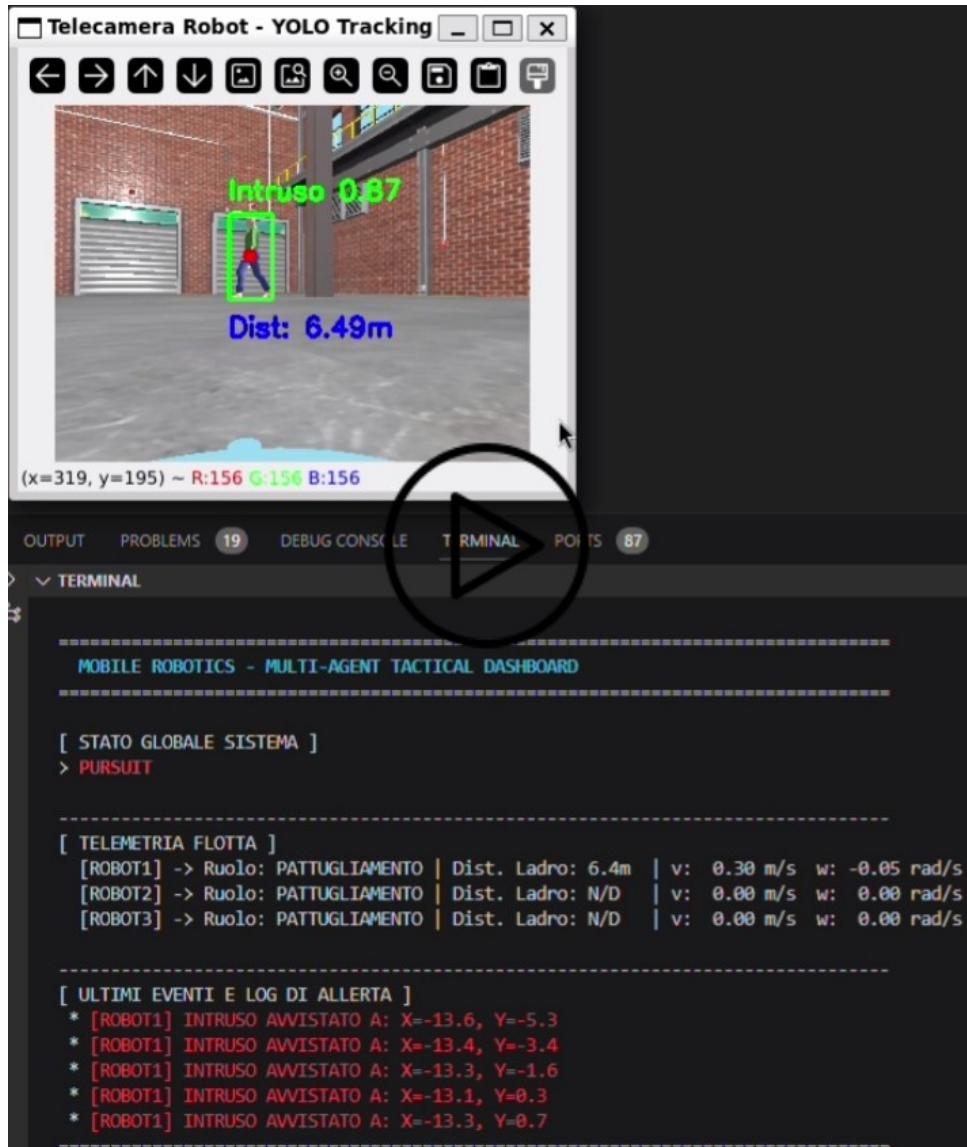


Figura 3.2: Visual demonstration of recognition through YOLO.

3.1.2 Detection and Pursuit

The footage demonstrates the architectural responsiveness of the system in the face of an intrusion. The upper portion of the screen shows the output of the `YOLO Tracking` node: the neural network identifies the intruder in real time, isolating him with a *bounding box* (green square) and estimating his linear metric distance relative to the camera lens ("Dist: ... m"). This visual perception triggers an immediate state change, visible in the *Tactical Dashboard* below: the global state transitions to "PURSUIT", logging the "INTRUDER SPOTTED" event with its relative spatial coordinates. Telemetry confirms the successful dynamic reassignment of tasks, with the robot switching to "PURSUIT" mode and updating its speeds, until it reaches the final "OBJECTIVE REACHED" state, which certifies the success of the interception maneuver.

3.2 Data Analysis

3.2.1 KPI1: Capture Time

Tactical Convergence Time: 197.40 seconds
(Pursuit start: 284.44 s | Objective reached: 481.84 s)

The system demonstrated solid operational responsiveness, triggering the pursuit at second 284.44 and closing the encirclement at second 481.84. Although this timeframe confirms the solidity of the target allocation algorithm, the time to reach the objective is influenced by the simulation conditions: the limited hardware resources significantly slowed down the speed of all the simulation's actors; at the same time, the use of an intruder without an evasive logic greatly facilitated the success of the defensive operation.

3.2.2 KPI2: Inter-robot Distance

The trajectory analysis confirms the effectiveness of the navigation system: the fleet constantly maintains distances above the critical safety threshold (0.35 m), dynamically reprogramming paths to avoid internal collisions. In addition to preventing physical interference, this natural spatial "repulsion" among the robots ensures a distributed and widespread coverage of the operating environment: by keeping the agents well spaced from each other during patrolling, the system maximizes the overall visual area of the fleet, drastically reducing blind spots and uncovered zones within the warehouse.

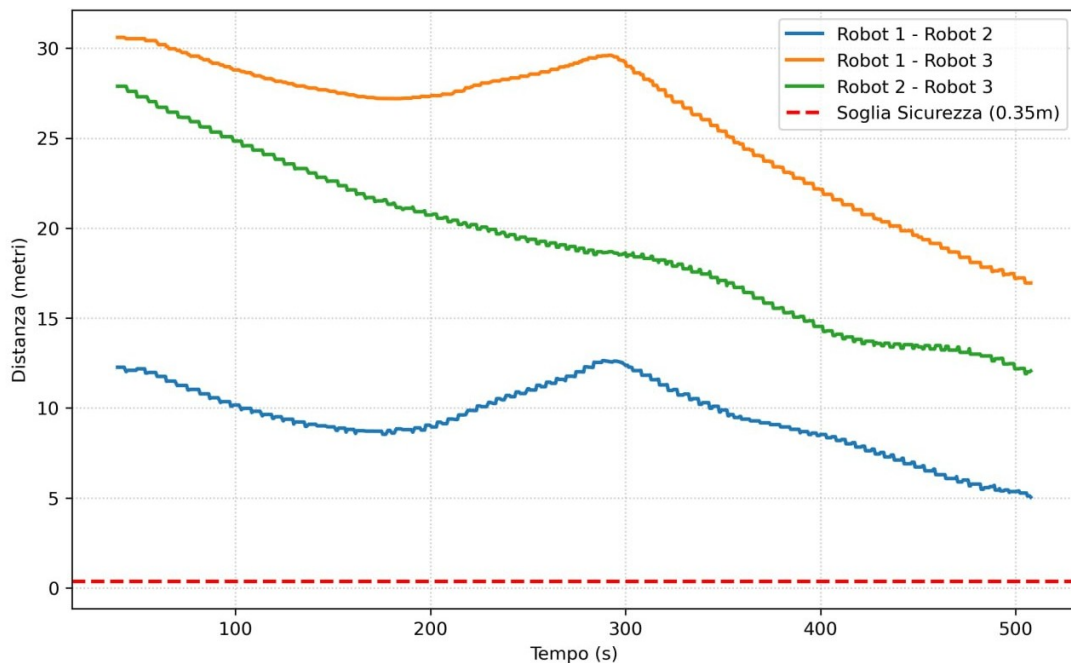


Figura 3.3: Inter-agent distance allowing for a safe, collision-free patrol.

3.2.3 KPI3: YOLO Model Accuracy

The computer vision *pipeline* recorded a high average confidence and notable stability (*uptime*) in keeping the *bounding box* locked onto the target during movement. It is important to emphasize that these metrics benefit from the perfect optical conditions provided by Gazebo's rendering: the uniform lighting of the warehouse and the total absence of *motion blur* or anomalous reflections offer the neural network an ideal scenario, which is difficult to replicate in a real industrial environment.

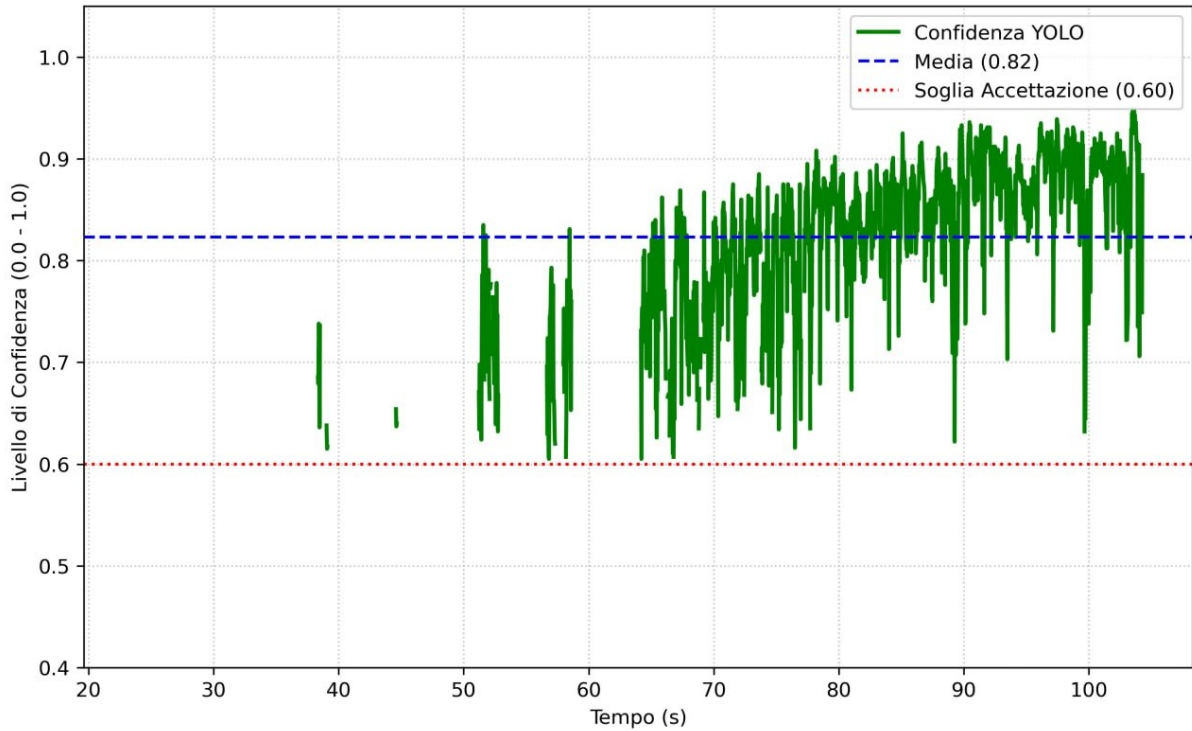


Figura 3.4: YOLO Visual Reliability: Average Detection Confidence at 82.32%; Tracking Stability (*Uptime*) at 52.2% of the crossing time.

4 | Conclusion

The project has achieved its intended goals, demonstrating the technical feasibility of a multi-agent autonomous patrolling system based on the integration of ROS2, Nav2, Gazebo, and computer vision algorithms. The tests conducted within the simulated perimeter confirmed that the fleet (*swarm*) is able to operate by decentralizing local kinematic control, while still obeying global tactical directives. The Turtlebots strictly maintained inter-agent safety distances (well above the critical *footprint* threshold of 0.35 m), avoiding cross-collisions and dynamically recalculating paths through the **MPPI** planner to converge on the target in operationally appropriate times. Furthermore, the integration of the YOLOv8 neural network with depth maps (*Depth Camera*) proved to be a robust solution: the inversion of the *Pinhole* optical model allowed for the effective projection of two-dimensional detections into real 3D space, enabling consistent kinematic tracking.

However, during the project's development, several critical issues arose, primarily due to a hardware bottleneck. The simultaneous presence of Gazebo, multiple navigation stacks (Nav2), and neural networks (YOLOv8) generated a particularly heavy computational load for the machines used for the development of the project itself. Moreover, the use of a purely passive intruder model (moving along predetermined trajectories and lacking evasive decision-making logic) prevented the extraction of high-level performance metrics (KPIs). The inability to simultaneously stress-test the entire architecture against a reactive threat effectively precluded a definitive performance analysis of the system under authentic operational load conditions.

4.1 Future Developments

In order to move past the current simulation phase and apply the project in real-world scenarios, the system's evolution will need to focus on four key areas:

- **Onboard Hardware Upgrade:** To overcome current computational limits, the use of higher-performing hardware directly onboard the robots is necessary.
- **Adaptive Intruder Behavior:** Currently, the simulated intruder model follows predefined trajectories. A significant improvement consists of equipping it with a navigation system capable of reacting dynamically to the fleet's behavior. By recognizing the robots as threats, the thief could modify its path in real time to attempt an escape, making the testing scenario much more realistic.
- **Predictive Interception:** To date, the swarm operates with a purely reactive pursuit logic. The future goal is to develop a predictive model that allows the robots to anticipate the thief's moves. By analyzing the warehouse map, the system will be able to coordinate the vehicles to strategically and "intelligently" block escape routes, encircling the intruder rather than just following its tracks.
- **Integration with External Security Systems:** The fleet's action must fit into a broader surveillance ecosystem. Upon threat detection, the system must be

able to interface with the building's infrastructure, automatically activating sirens, alarms, and fixed cameras. The aid of the robots will therefore serve as a trigger for environmental countermeasures and to promptly transmit video streams to a security control room supervised by human operators.